RAFFLES INSTITUTION
H2 Computing (9569)

2025 Year 5

Chapter S1B: Searching Algorithms

CONTENT

1. INTRODUCTION TO SEARCHING

Searching the elements of an array or data structure for a data item is a common operation. The reasons for searching an array may be:

- to check that it is a valid element of an array,
- to edit or delete an element in an array,
- to retrieve information from an array, or
- to retrieve information from a corresponding element in a paired array.

A sorted array is advantageous for searching. Either a linear search or a binary search may be performed.

2. LINEAR SEARCH

A search algorithm that examines the elements of a sequence (i.e.: or any other data structures), **starting from a chosen element** of the sequence and subsequently **moving on item by item, without the possibility of skipping,** until the desired element is found or not found (i.e.: when all elements of sequence are examined).

Process: Locating a particular element in a given sequence.
Input: Search key (target) and data set (sequence).
Output: Result/outcome of the search.

2.1 On an Unordered Array

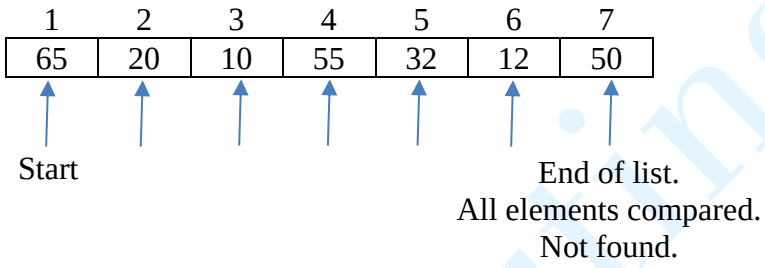
The algorithm in pseudocode:

```
PROCEDURE LinearSearchOnArray (searchKey, array)
    Set max_num_elements to length of array
    Set element_found to False
    //This flag, initially set to false, will be set to true
    //once the value being looked for is found, that is,
    //when current element of the array is equal to the
    //item being looked for.
    Set index to 1
    WHILE (NOT element_found) AND (index <= max_num_elements)
        IF array[index] = searchKey THEN
            Set element_found to True
            Exit loop
        ELSE
            index ← index + 1
        ENDIF
    ENDWHILE
    IF element_found = True THEN
        Print array[index]
    ELSE
        Print "value not found", searchKey
    ENDIF
ENDPROCEDURE
```

The algorithm described in steps:

1. Read the search key (item to be searched) from user.
2. Compare search key with the first element of the array.
3. If both match, return element found and terminate search.
4. If both do not match, move on to next element in array and repeat step 3 and 4, until last element is compared.
5. If search key and last element still do not match, return element not found and terminate search.

Example: searchKey: 37



Note: As elements are NOT in order, any element can be found anywhere. Hence, every element in the array has to be examined until it is found (or not found).

2.2 On an Ordered Array

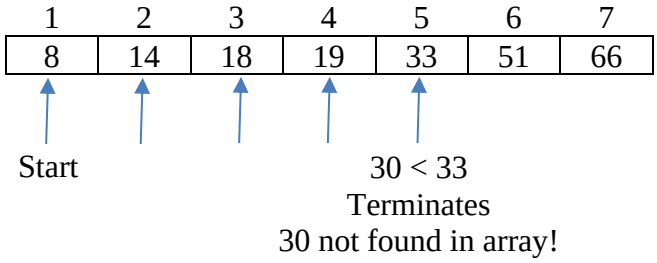
The algorithm in pseudocode:

```
FUNCTION LinearSearchOnArray (searchKey: INTEGER, arr: ARRAY) RETURNS BOOLEAN
    //data in arr is sorted in ascending order
    FOR i = 1 TO LENGTH (arr) // LENGTH() returns number of elements in arr
        IF arr[i] = searchKey THEN
            RETURN True
        ELSE IF searchKey < arr[i] THEN
            RETURN False
        ENDIF
    ENDFOR
    RETURN False
ENDFUNCTION
```

The algorithm described in steps:

1. Read the search key (item to be searched) from user.
2. Compare search key with the first element of the array.
3. If both match, return element found and terminate search.
4. IF search key < next element, return element not found and terminate search.
5. If search key > next element, move on to next element and repeat step 3,4,5 until last element is compared.
6. If search key and last element still do not match, return element not found and terminate search.

Example: searchKey: 30



2.3 Recursive Algorithm

Three approaches of using recursion for linear search are shown here.

Approach 1: Compare search key with the **first element** in array. If an element is found at the first position, return the index. Else call recursive function with array (minus first element) and search key.

```
FUNCTION recSearch1(arr, first, searchKey)://search from first element
    IF first >= LENGTH(arr) THEN      //base case: array size exceeded
        RETURN FALSE
    ELSE IF arr[first] = searchKey THEN   //searchKey found
        RETURN first
    ENDIF
    RETURN recSearch1(arr, first+1, searchKey)//search from next element
ENDFUNCTION
```

Approach 2: Compare search key with the **last element** in array. If an element is found at the last position, return the index. Else call recursive function with array (minus last element) and search key.

```
FUNCTION recSearch2(arr, last, searchKey)://start search from last element
    IF last < 1 THEN          //base case: index less than first element
        RETURN FALSE
    ELSE IF arr[last] = searchKey THEN //searchKey found
        RETURN last
    ENDIF
    RETURN recSearch2(arr, last-1, searchKey)//search from previous element
ENDFUNCTION
```

Approach 3: For each function call, compare search key with the **first** and then **last element** in array. If an element is found at first or last position, return the index. Else call recursive function with array (minus first and last element) and search key.

```
FUNCTION recSearch3(arr, first, last, searchKey)://search from both ends
    IF first > last THEN     //base case: complete scan of all elements
        RETURN False
    ELSE IF arr[first] = searchKey THEN //searchKey found at first index
        RETURN first
    ELSE IF arr[last] = searchKey THEN //searchKey found at last index
        RETURN last
    ENDIF
    RETURN recSearch3(arr, first+1, last-1, searchKey)
ENDFUNCTION
```

2.4 Time Complexity

For search algorithms, the main steps are the comparisons of list values with the target value. Counting these for data models representing the **best case**, the **worst case**, and the **average case** produces the following table. For each case, the number of steps is expressed in terms of n, the number of items in the list.

Model	Number of Comparisons (for n = 100000)	Comparisons as a function of n	Time complexity
Best Case (fewest comparisons)	1 (target is first item)	1	O(1)
Worst Case (most comparisons)	100000 (target is last item)	n	O(n)
Average Case (average number of comparisons)	50000 (target is middle item)	n/2	O(n)

The **best-case** time complexity is O(1) where the element is found at the first index. If the list grows in size, the number of comparisons required to find a target item in both worst and average cases grow linearly. In general, for a list of length n, the **worst case** is n comparisons, hence O(n) time complexity. The algorithm is called linear search because its complexity/efficiency can be expressed as a linear function: the number of comparisons to find a target increases linearly as the size of the list.

3. BINARY SEARCH

Binary search is a divide and conquer algorithm, which breaks down a problem into smaller sub-problem of the same type, until it becomes simple enough to be solved directly. The solutions to the sub-problems are then combined to give a solution to the original problem.

A **pre-condition** for binary search is that **the sequence is ordered** or **data set is sorted in a pre-defined order**.

Binary search locates the middle element of the array, compares it with the search key, to determine if the search key is in the left subarray or right subarray. The search then locates the middle element of the relevant subarray, and the comparison is repeated. This technique of **comparing and continually halving the data set** searched is continued until the search key is found, or its absence is detected.

The algorithm described in steps:

1. Find the middle element.
2. If the middle element is the search key, return element found.
3. If search key is smaller than the middle element, repeat from step 1 using the sub-array ordered before the middle element.
4. Else, repeat from step 1 using the sub-array ordered after the middle element.
5. This process continues on until the size of the sub-array reduces to 0, which means element is not found.

searchKey 24
Middle element is 40

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
	8	16	24	32	40	48	56	64	72

24<40, search lower half

	low				mid				high
	8	16	24	32	**40**	48	56	64	72

24>16, search upper half

	low	mid		high					
	8	**16**	24	32	40	48	56	64	72

Found 24, Return True

			low,mid	high					
	8	16	**24**	32	40	48	56	64	72

3.1 Iterative Algorithm

```
FUNCTION BinarySearch (searchKey: INTEGER, arr: ARRAY) RETURNS BOOLEAN
    // data in arr is sorted in ascending order
    DECLARE low, high, mid: INTEGER
    low ← 1                 // 1-based indexing
    high ← LENGTH(array)    // LENGTH() returns the number of elements

    WHILE low <= high DO
        mid ← INT((low + high)/2)      //INT() removes decimal part
        IF searchKey = arr[mid]  THEN      //searchKey found
            RETURN TRUE
        ELSE IF searchKey < arr[mid] THEN //search lower half next
            high ← mid - 1
        ELSE
            low ← mid + 1                  //search upper half next
        ENDIF
    ENDWHILE
    RETURN FALSE                           //searchKey not found
ENDFUNCTION
```

3.2 Recursive Algorithm

```
FUNCTION BinarySearchRec(low, high, key: INTEGER, arr: ARRAY)
                         RETURNS BOOLEAN
    // data in arr is sorted in ascending order
    IF low > high  THEN           //base case 1: not found
        RETURN FALSE
    ELSE
        DECLARE mid: INTEGER
        mid ← INT((low + high)/2) //INT() removes decimal part
        IF key = arr[mid]  THEN   //base case 2: found
            RETURN TRUE
        ELSE IF key < arr[mid] THEN  //search lower half next
            RETURN BinarySearchRec(low, mid-1, key, arr)
        ELSE                         //search upper half next
            RETURN BinarySearchRec(mid + 1, high, key, arr)
        ENDIF
    ENDIF
ENDFUNCTION
```

3.3 Time Complexity

The best case occurs if the middle item happens to be the target. Then only one comparison is needed to find it.

When does the worst case occur? If the target is not in the array then the process of dividing the list in half continues until there is only one item left to check. Here is the pattern of the number of comparisons after each division, given the simplifying assumptions of an initial array length that's an even power of 2 (1024) and exact division in half on each iteration:

Items Left to Search	Comparisons made
1024	0
512	1
256	2
128	3
64	4
32	5
16	6
8	7
4	8
2	9
1	10

For a list size of 1024, *there are 10 comparisons to reach a list of size one*, given that there is one comparison for each division, and each division splits the size of list in half.

In general, if **n** is the size of the list to be searched and **C** is the number of comparisons to do so in the worst case, **$C = \log_2 n$**. Thus, the efficiency of binary search can be expressed as a logarithmic function, in which the number of comparisons required to find a target increases only logarithmically with the size of the list.

The following table summarises the analysis for binary search.

Model	**Number of Comparisons** (for n = 100000)	**Comparisons as a function of n**	**Time complexity**
Best Case (fewest comparisons)	1 (target is middle item)	1	O(1)
Worst Case (most comparisons)	17 (target not in array)	$\log_2 n$	$O(\log_2 n)$

4. Comparison of Linear and Binary Search

	Advantages	Disadvantages
Linear search	• Additional data items can be simply appended to the end of the list. • Algorithm is straightforward and easy to implement. • Works well with a small number of data items.	• Becomes inefficient with a large number of data items. • Every item must be checked until the target is found or the list ends (especially inefficient if the item is missing).
Binary search	• Efficient for searching large amounts of data. • Missing data items can be identified quickly (e.g. search fails faster).	• Requires the data to be sorted before the search. • Inserting or deleting items is slower as the list may need to be re-sorted.

5. Video References

Linear search
https://www.youtube.com/watch?v=TwsgCHYmbbA

Binary search
https://www.youtube.com/watch?v=T98PIp4omUA

Tutorial S1B: Searching Algorithms

Section A (Discussion Questions)

1. The names of 25 students in a Computing class are to be stored in an array. If the array was large enough to store the names of 2000 students in the school, explain why a linear search would not be as suitable to search for a particular student by name, compared to a binary search.

2. Dry-run the binary search (non-recursive) algorithm using a trace table. Assume the list consists of the following 21 items in the order given:
   4, 9, 20, 24, 30, 34, 38, 42, 53, 56, 58, 60, 68, 71, 79, 82, 85, 90, 93, 95, 99.
   (a) Search for the value 60. How many times was the while loop executed?
   (b) Dry-run the algorithm again, this time searching for the value 35. How many times was the while loop executed?

3. **[Modified from 9348/1997/01/07]**
   A program is being designed to handle a list of about 10000 names held in main store, with a fixed number of store locations for each name. Two suggestions have been made concerning the organisation of this data:
   - The names could be kept in alphabetical order of surname.
   - The names could be kept in an unsorted array.

   During the running of the program the following four operations will be needed.
   - To search for a particular name.
   - To insert new names.
   - To delete a name which has already been located.
   - To print the whole list in alphabetical order of surname.

   (a) Comment on the speed with which each of these operations could be carried out with each of the two possible data structures.
   (b) Describe the criteria you would consider in choosing which of these two data structures to use for a particular application.

Section B (Assignment Questions)

1. Sam uses two 1D arrays to store the userIDs and passwords for a group of 20 users. He stores userIDs in `userList` and the corresponding user's password in `passwordList`. For example, he stores `nick12` in index 2 of `userList` and the password for `nick12` in index 2 of `passwordList`.
   Sam wants to write an algorithm to check whether a userID and password, entered by a user, are correct. He design the algorithm to search linearly in `userList` for the userID. If the password is found, the password stored in `passwordList` is to be compared to the entered password. If the passwords match, the login is successful. Otherwise, the login is unsuccessful.

(a) Complete the identifier table by completing the six blanks.

Identifier	Description
`userList[1..20]`	1D array to store userIDs
`(1)`.....	1D array to store passwords
`maxIndex`	`(2)`.....
`myUserID`	userID entered to login
`myPassword`	`(3)`.....
`userIDFound`	False if userID not found in userList True if `(4)`.....
`loginOK`	False if `(5)`..... True if `(6)`.....
`index`	Pointer to current array element

(b) Complete the pseudocode for Sam's algorithm by completing A to J:

```
maxIndex ← 20
INPUT myUserID
INPUT myPassword
userIDFound ← FALSE
loginOK ← .....A.....
index ← 0
REPEAT
    index ← .....B.....
    IF userList[.....C.....] = .....D.....  THEN
        userIDFound ← TRUE
    ENDIF
UNTIL .....E..... OR .....F.....
IF userIDFound = TRUE  THEN
    IF passwordList[.....G.....] = .....H.....  THEN
        loginOK ← .....I.....
    ENDIF
ENDIF
IF .....J.....  THEN
    OUTPUT "Login successful"
ELSE
    OUTPUT "UserID and/or password incorrect"
ENDIF
```

2. A list of positive odd numbers up to 1000 is 1, 3, 5, …, 499, 501, …, 995, 997, 999. An attempt is made to search for the number 888 using linear search and binary search.
(a) Show the first few stages of comparisons to search for the number 888.
(b) Explain the difference between linear searching and binary searching, using the above search for number 888 as an example.
(c) State one advantage and one disadvantage of a binary search compared with a linear search.